

Présentation Projet Programmation Impérative

Percet Gauthier Nikita Volianskyi Mark Melkonian

- 1 Introduction
- 2 Architecture du code
- 3 Analyse des graphiques
- 4 Analyse
- 5 Conclusion

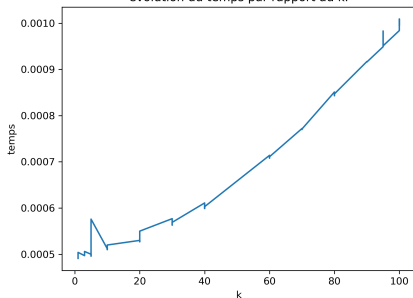
- Dans le cadre du projet de programmation impérative, nous avons réalisé une implémentation de l'algorithme des K plus proches voisins (KPPV).
- L'objectif de ce projet était de réaliser un programme complet permettant d'exécuter l'algorithme des KPPV avec deux types de structures de données.
- Nous avons utilisé pour cela le langage C, la bibliothèque MLV, Python et Bash.

Structure du projet

- **main.c** : Point d'entrée, mesures de performance et sauvegarde des résultats.
- **interface.c** : Gestion de l'affichage graphique (MLV).
- **tree_kd.c** : Implémentation de la recherche KPPV par arbre KD.
- **points_list.c** : Implémentation de la recherche KPPV par liste (force brute).
- **vector.c / point.c** : Structures de données de base.
- **test.sh / graphique.py** : Automatisation des tests et génération des graphiques.

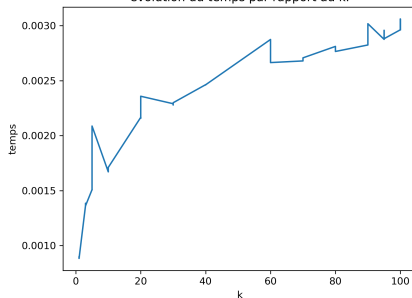
Variation de k (liste vs arbre)

evolution du temps par rapport au k.



algorithme liste

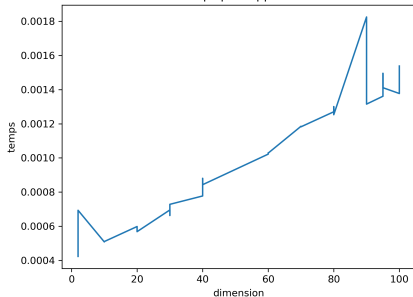
evolution du temps par rapport au k.



algorithme arbre

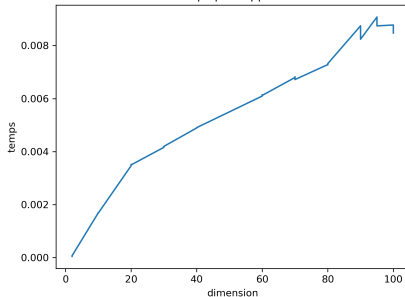
Variation des dimensions (liste vs arbre)

evolution du temps par rapport au dimension.



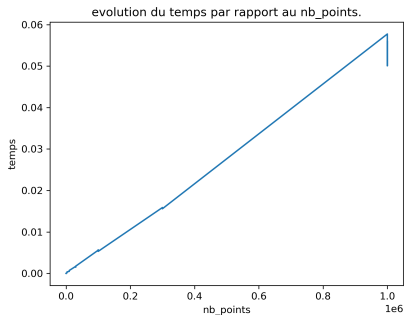
algorithme liste

evolution du temps par rapport au dimension.

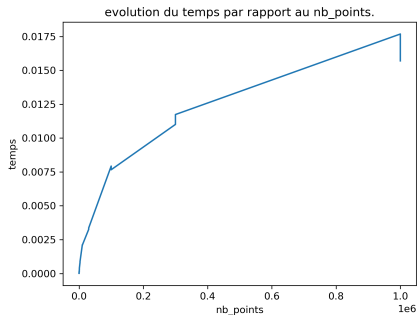


algorithme arbre

Variation du nombre de points (liste vs arbre)



algorithme liste



algorithme arbre

- variation de k voisins :
 - liste : évolution exponentielle
 - arbre : évolution logarithmique
- variation de dimension :
 - liste : évolution exponentielle
 - arbre : évolution logarithmique
- variation du nombre de point :
 - liste : évolution continue
 - arbre : évolution logarithmique

Conclusion

- Bilan des fonctionnalités implémentées.
- Difficultés rencontrées et solutions apportées.
- Perspectives d'amélioration.